



Boletín 03 Extras — Retos Algorítmicos y Competitivos

Retos de alta intensidad algorítmica inspirados en plataformas de programación competitiva (CodeWars, Acepta el Reto, LeetCode) para la Unidad 3 en Java 25 LTS.

Bienvenido al boletín de retos competitivos de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Los siguientes 6 desafíos plantean problemas de optimización computacional, complejidad temporal y espacial, manipulación avanzada de memoria y algoritmos clásicos de la informática. Cada reto incluye su planteamiento formal, análisis de complejidad, pistas estratégicas y la solución íntegra comentada en Java 25 LTS.

Reto 1: La Criba de Eratóstenes (Generador de Números Primos)

Contexto y Planteamiento

Encontrar todos los números primos hasta un límite N comprobando la primalidad de cada número uno por uno mediante divisiones sucesivas tiene una complejidad temporal inasumible $O(N \times \sqrt{N})$. El sabio griego Eratóstenes diseñó un método de filtrado que tacha sistemáticamente los múltiplos de cada primo encontrado.

Implementa un método `public static int[] cribaEratostenes(int n)` que devuelva un array con todos los números primos menores o iguales a n .

- **Complejidad requerida:** $O(N \log \log N)$ en tiempo.
- **Pista algorítmica:** Usa un array booleano `esPrimo` de tamaño $n + 1$ inicializado a `true`. Empieza en $p = 2$; tacha todos sus múltiplos comenzando directamente desde $p \times p$. El bucle exterior solo necesita iterar hasta $\sqrt{n}$.

►  Ver Solución Comentada

Reto 2: Compresión de Cadenas por Racha (Run-Length Encoding)

Contexto y Planteamiento

El algoritmo **RLE** es una técnica clásica de compresión de datos sin pérdida. Consiste en sustituir secuencias repetidas consecutivas del mismo carácter por el número de repeticiones seguido del carácter en cuestión.

Ejemplo: La cadena `"AAAABBBCCDAA"` se comprime como `"4A3B2C1D2A"`.

Diseña dos métodos complementarios:

1. `public static String comprimirRLE(String texto)`
2. `public static String descomprimirRLE(String textoComprimido)`

- **Requisito:** Utiliza `StringBuilder` para garantizar una complejidad lineal $O(N)$ en tiempo y evitar la sobrecarga del Garbage Collector.

►  Ver Solución Comentada

Reto 3: El Número Perdido en Tiempo Lineal y Memoria O(1)

Contexto y Planteamiento

Se te proporciona un array desordenado de tamaño $N - 1$ que contiene números enteros del 1 al N sin ningún duplicado, excepto **exactamente un número que ha desaparecido**.

Diseña un método `public static int encontrarFaltante(int[] arr, int n)` que localice el número omitido cumpliendo:

- Complejidad Temporal: $O(N)$.
- Espacio Adicional: $O(1)$ (está prohibido crear nuevos arrays o estructuras intermedias).
- **Pista matemática:** La suma de los primeros N números naturales viene dada por la fórmula de Gauss $Suma = N \times (N + 1) / 2$. La diferencia entre la suma teórica y la suma real de los elementos del array es el número que falta.

►  Ver Solución Comentada

Reto 4: Rotación Cíclica de un Vector In-Place (Algoritmo de las Tres Inversiones)

Contexto y Planteamiento

Dado un array de enteros y un valor k , rota los elementos del array hacia la derecha k posiciones.

Ejemplo: Si `arr = {1, 2, 3, 4, 5, 6, 7}` y `k = 3`, el array resultante debe ser `{5, 6, 7, 1, 2, 3, 4}`.

- **Requisito estricto:** El algoritmo debe modificar el array in-place, con memoria adicional $O(1)$ y tiempo $O(N)$.
- **Estrategia (Las 3 Inversiones):**
 1. Invierte el array completo: `{7, 6, 5, 4, 3, 2, 1}`.
 2. Invierte los primeros k elementos: `{5, 6, 7, 4, 3, 2, 1}`.
 3. Invierte los elementos restantes desde k hasta el final: `{5, 6, 7, 1, 2, 3, 4}`.

►  Ver Solución Comentada

Reto 5: Las Torres de Hanói (Recursividad Pura)

Contexto y Planteamiento

Las Torres de Hanói es un rompecabezas clásico donde se deben trasladar N discos de radios decrecientes desde la varilla **Origen** hasta la varilla **Destino**, utilizando una varilla **Auxiliar** intermedia.

- Solo se puede mover un disco por turno.
- Ningún disco mayor puede descansar sobre uno menor en ningún momento.

Diseña un método recursivo `public static void resolverHanoi(int n, char origen, char auxiliar, char destino)` que imprima cada movimiento elemental y devuelva la cantidad total de movimientos ($2^N - 1$).

►  Ver Solución Comentada

Reto 6: Validador de Delimitadores Equilibrados con Simulación de Pila

Contexto y Planteamiento

En compiladores e intérpretes, es crucial verificar que los delimitadores de apertura `(`, `[`, `{` se cierran en el orden correcto con sus respectivos cierres `)`, `]`, `}`.

Ejemplos:

- `"{ [ ( ) ] }"` -> **Válido** (anidamiento perfecto).
- `"( [ ) ]"` -> **Inválido** (cruce de delimitadores).
- `"((("` -> **Inválido** (delimitadores sin cerrar).

Implementa un método `public static boolean sonDelimitadoresValidos(String codigo)` empleando un array de caracteres `char[]` para simular internamente la estructura de una pila (LIFO) y un puntero de tope `top`.

►  Ver Solución Comentada